EDUHUB / ENGINEERING DOCUMENTATION

COMPONENT DIAGRAM

EduHub | Online Learning Platform

01

WEB COMPONENTS

MVC controllers, Razor UI and Identity

02

BUSINESS SERVICES

Modular use cases and service contracts

03

DATA & ADAPTERS

EF Core, Stripe, Blob and PDF rendering

MODEL SCOPE

9 logical UML component views, with interfaces, dependencies, module ownership, integration adapters and cross-document traceability. The three project assemblies remain one deployable ASP.NET Core application.

SOURCE EduHub project brief + existing architecture and database/class models

SYSTEM ANALYSIS & DESIGN / 05 NOVEMBER 2026

01 | Component design baseline

Purpose, architectural context, evidence classification and document boundary.

ARCHITECTURAL DECISION

N-TIER MODULAR MONOLITH

EduHub.Web owns presentation; EduHub.Services owns use cases and policies; EduHub.Data owns persistence and proposed external adapters. These components compile into one ASP.NET Core MVC deployment. The database and Stripe/Blob providers remain separately operated services. [S1, S2, S4]

EVIDENCE STATUS

STATUS	MEANING	EXAMPLES
BRIEF-REQUIRED	Explicit in the original 5-page project brief	Three projects, MVC, Identity, EF Core, Repository / UoW, Strategy / Observer, Azure Blob, Stripe, PDF.
ESTABLISHED	Design already modeled in prior EduHub deliverables	38 logical entities; 3-layer monolith; Azure SQL proposed; 48 use cases; dependency abstractions.
DESIGN PROPOSAL	Component names / adapters chosen to make a buildable solution	Named service interfaces, controller grouping, webhook service, media processor port.
OPTIONAL	Not required for the graduation MVP	Dedicated worker, 4th Infrastructure project, distributed broker, special security tier.

SCOPE BOUNDARY

A component diagram shows a system assembled from encapsulated modules, their provided/required interfaces, and dependencies. It is not a database ERD, runtime deployment diagram, sequence diagram or executable API contract. The specific C# signatures and endpoint payloads belong in the later API design deliverable.

02 | UML component notation

How to read the nine views and avoid mixing UML diagram types.

DIAGRAM VOCABULARY

NOTATION	INTERPRETATION	EXAMPLE
«component»	Replaceable or cohesive logical part with public interactions.	Commerce service / MVC Areas.
«interface»	Contract offered to consuming components and realized by implementations.	ICheckoutService, IMediaStorage.
«external»	Boundary managed outside the EduHub application.	Stripe, Azure SQL, Azure Blob.
Dependency arrow	A component invokes, reads or requires another contract.	MVC controller → ICourseService.
Dashed realization	Implementing component satisfies a declared interface.	StripeGateway → IPaymentGateway.
Package boundary	Module or project scope; not a deployed server.	EduHub.Web / Services / Data.
Status label	Proposed implementation detail rather than source requirement.	Media processing provider to be selected.

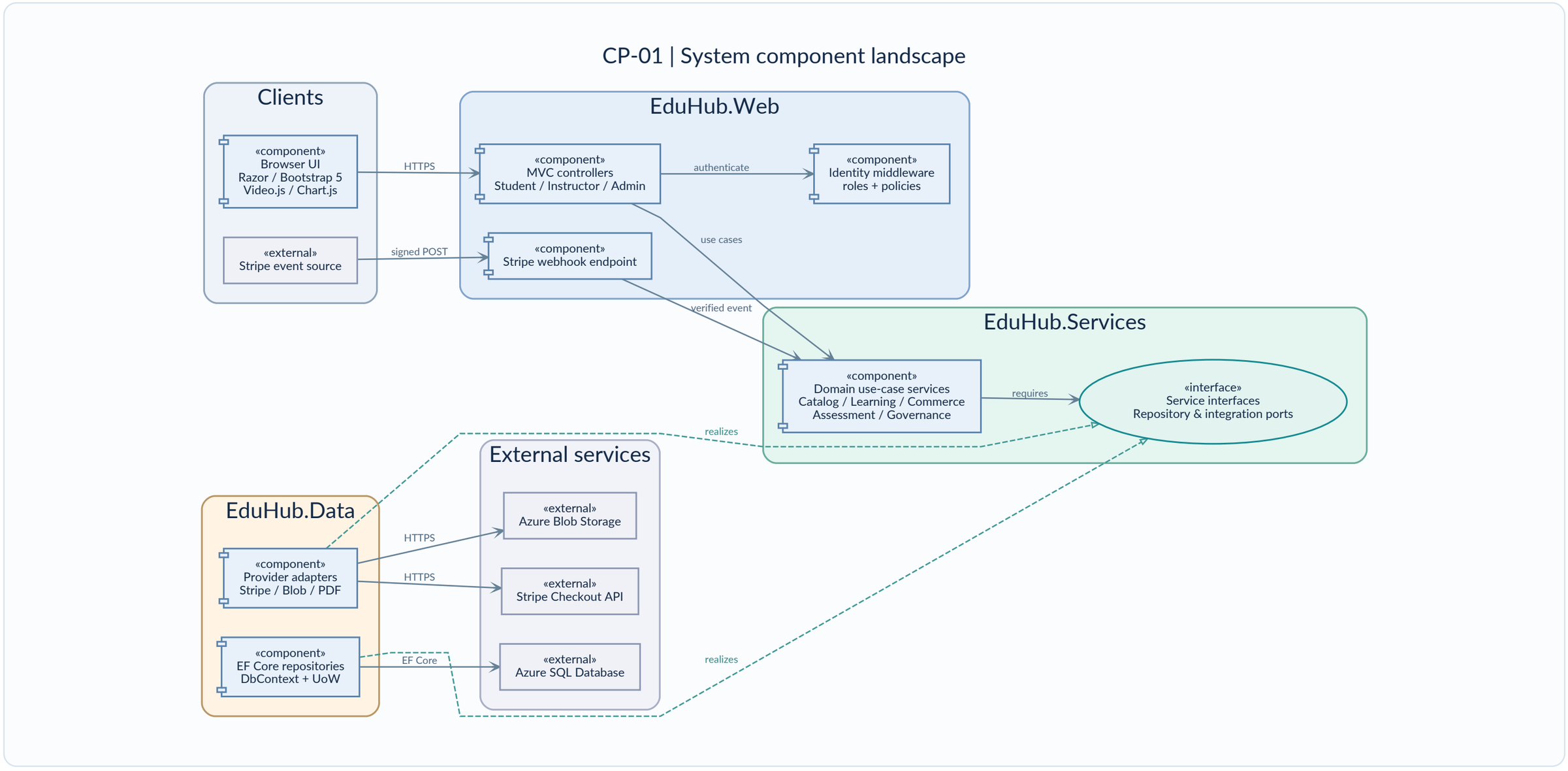
TWO KINDS OF DEPENDENCIES

Runtime calls typically flow Browser → Web → Services → persistence/integration adapters. Compile-time dependencies follow contract ownership: Services defines provider-neutral ports; Data implements them; Web can reference Data at the composition root to register adapters. The service layer should not reference provider SDKs directly.

> **UML INTERPRETATION**
>
> The graphical views use component-shape boxes and explicit interface nodes with labeled realization dependencies. They are UML-aligned conceptual views. The editable PlantUML files preserve the same nodes and links; Graphviz DOT is the canonical rendered source. [R1]

CP-01 | System component landscape

One deployment unit; collaborating internal modules and managed external systems.



READING THE COMPONENT VIEW

- Solid connectors show calls or data access; dashed realization connectors tie implementations to stable service-side contracts.
- The 3 projects are logical layers in a single MVC deployment. Stripe, Azure SQL and Blob are operated separately.

Baseline: N-Tier modular monolith. Source brief specifies Web/Services/Data and the three roles; adapter packaging is a recommendation.

CP-01 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
Clients	Browser UI / Razor / Bootstrap 5 / Video.js / Chart.js	COMPONENT
Clients	Stripe event source	COMPONENT
EduHub.Web	MVC controllers / Student / Instructor / Admin	COMPONENT
EduHub.Web	Identity middleware / roles + policies	COMPONENT
EduHub.Web	Stripe webhook endpoint	COMPONENT
EduHub.Services	Domain use-case services / Catalog / Learning / Commerce / Assessment / Governance	COMPONENT
EduHub.Services	Service interfaces / Repository & integration ports	PORT
EduHub.Data	Provider adapters / Stripe / Blob / PDF	COMPONENT

INTERACTION INVARIANTS

- Solid connectors show calls or data access; dashed realization connectors tie implementations to stable service-side contracts.
- The 3 projects are logical layers in a single MVC deployment. Stripe, Azure SQL and Blob are operated separately.

IMPLEMENTATION GUIDANCE

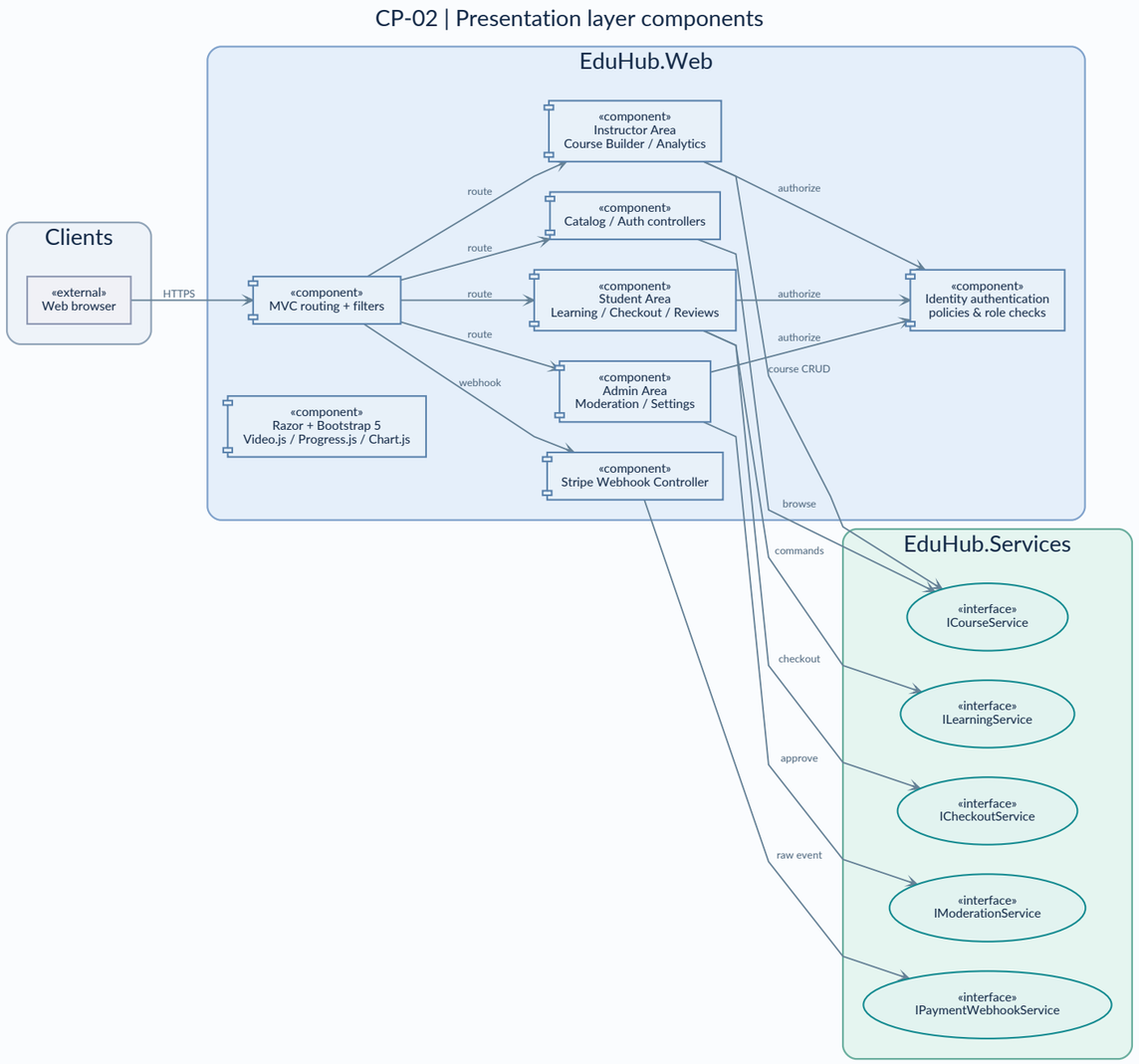
Baseline: N-Tier modular monolith. Source brief specifies Web/Services/Data and the three roles; adapter packaging is a recommendation.

MODELED EDGES

SOURCE	TARGET	ROLE
Browser UI	MVC controllers	HTTPS
MVC controllers	Identity middleware	authenticate
MVC controllers	Domain use-case services	use cases
Stripe webhook endpoint	Domain use-case services	verified event
Stripe event source	Stripe webhook endpoint	signed POST

CP-02 | Presentation layer components

ASP.NET Core MVC entry points, authorization and role-based UI surfaces.



READING THE COMPONENT VIEW

- MVC controllers handle transport concerns, model binding and response composition, not pricing, persistence or entitlement rules.
- Webhook signature checks happen at the trusted server-side boundary; a payment success redirect is not fulfillment authority.

Some controller/interface names are recommended logical components, not names fixed by the course brief.

CP-02 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
Clients	Web browser	COMPONENT
EduHub.Web	MVC routing + filters	COMPONENT
EduHub.Web	Catalog / Auth controllers	COMPONENT
EduHub.Web	Student Area / Learning / Checkout / Reviews	COMPONENT
EduHub.Web	Instructor Area / Course Builder / Analytics	COMPONENT
EduHub.Web	Admin Area / Moderation / Settings	COMPONENT
EduHub.Web	Stripe Webhook Controller	COMPONENT
EduHub.Web	Identity authentication / policies & role checks	COMPONENT

INTERACTION INVARIANTS

- MVC controllers handle transport concerns, model binding and response composition, not pricing, persistence or entitlement rules.
- Webhook signature checks happen at the trusted server-side boundary; a payment success redirect is not fulfillment authority.

IMPLEMENTATION GUIDANCE

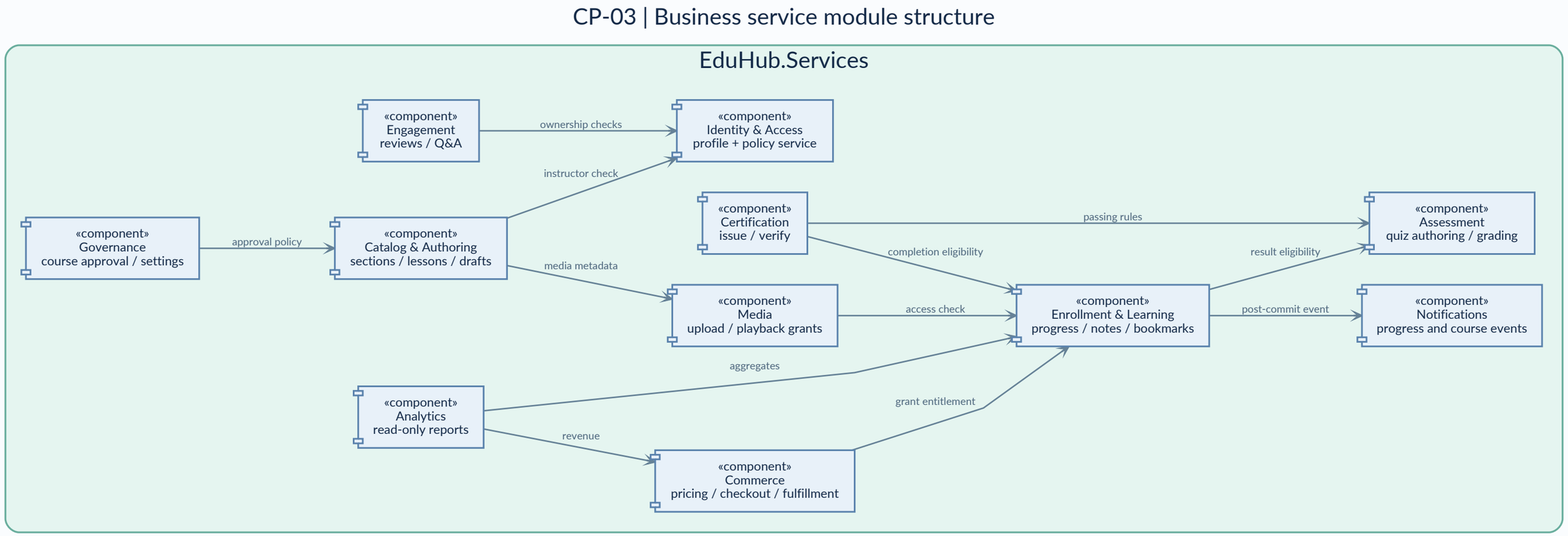
Some controller/interface names are recommended logical components, not names fixed by the course brief.

MODELED EDGES

SOURCE	TARGET	ROLE
Web browser	MVC routing + filters	HTTPS
MVC routing + filters	Catalog / Auth controllers	route
MVC routing + filters	Student Area	route
MVC routing + filters	Instructor Area	route
MVC routing + filters	Admin Area	route

CP-03 | Business service module structure

Cohesive functional boundaries inside EduHub.Services.



READING THE COMPONENT VIEW

- Modules are component boundaries, not independent executables, databases or network services.
- Cross-module writes must occur through another module contract or a designated orchestration service; reports use authorized read models.

Suggested interfaces may evolve, but policy ownership should remain cohesive within this module layout.

CP-03 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
EduHub.Services	Identity & Access / profile + policy service	COMPONENT
EduHub.Services	Catalog & Authoring / sections / lessons / drafts	COMPONENT
EduHub.Services	Governance / course approval / settings	COMPONENT
EduHub.Services	Media / upload / playback grants	COMPONENT
EduHub.Services	Commerce / pricing / checkout / fulfillment	COMPONENT
EduHub.Services	Enrollment & Learning / progress / notes / bookmarks	COMPONENT
EduHub.Services	Assessment / quiz authoring / grading	COMPONENT
EduHub.Services	Certification / issue / verify	COMPONENT

INTERACTION INVARIANTS

- Modules are component boundaries, not independent executables, databases or network services.
- Cross-module writes must occur through another module contract or a designated orchestration service; reports use authorized read models.

IMPLEMENTATION GUIDANCE

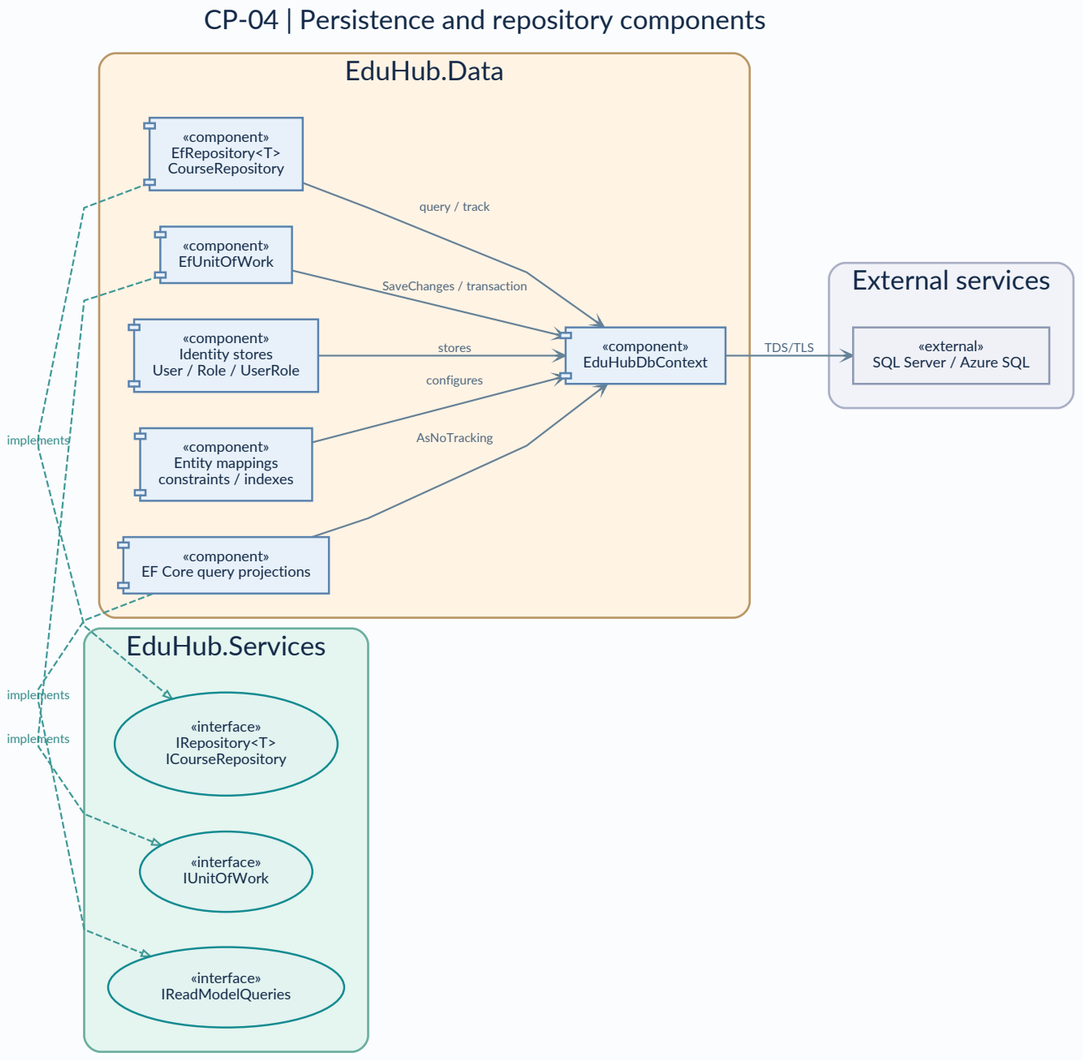
Suggested interfaces may evolve, but policy ownership should remain cohesive within this module layout.

MODELED EDGES

SOURCE	TARGET	ROLE
Catalog & Authoring	Media	media metadata
Governance	Catalog & Authoring	approval policy
Commerce	Enrollment & Learning	grant entitlement
Enrollment & Learning	Assessment	result eligibility
Enrollment & Learning	Notifications	post-commit event

CP-04 | Persistence and repository components

EF Core, ASP.NET Core Identity, repositories and Unit of Work.



READING THE COMPONENT VIEW

- Data implements persistence ports defined outside its concrete EF Core classes; the business layer does not require provider types.
- One DbContext-backed unit of work commits related changes. Unique constraints protect enrollment and webhook deduplication.

The SQL schema retains 38 entities: 35 EduHub domain tables plus the previously modeled Identity tables.

CP-04 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
EduHub.Services	IRepository / ICourseRepository	PORT
EduHub.Services	IUnitOfWork	PORT
EduHub.Services	IReadModelQueries	PORT
EduHub.Data	EfRepository / CourseRepository	COMPONENT
EduHub.Data	EfUnitOfWork	COMPONENT
EduHub.Data	EduHubDbContext	COMPONENT
EduHub.Data	Identity stores / User / Role / UserRole	COMPONENT
EduHub.Data	Entity mappings / constraints / indexes	COMPONENT

INTERACTION INVARIANTS

- Data implements persistence ports defined outside its concrete EF Core classes; the business layer does not require provider types.
- One DbContext-backed unit of work commits related changes. Unique constraints protect enrollment and webhook deduplication.

IMPLEMENTATION GUIDANCE

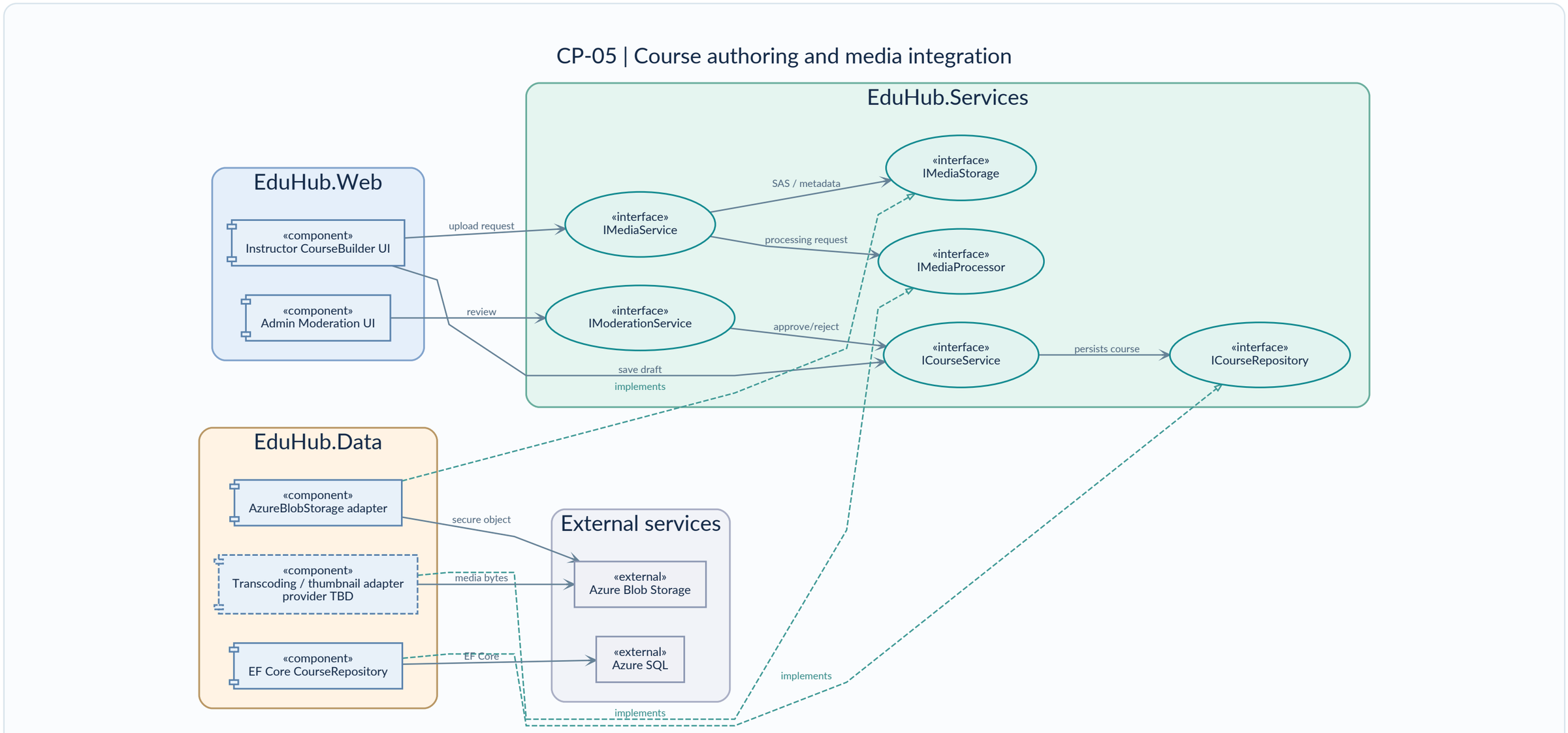
The SQL schema retains 38 entities: 35 EduHub domain tables plus the previously modeled Identity tables.

MODELED EDGES

SOURCE	TARGET	ROLE
EfRepository	IRepository	implements
EfUnitOfWork	IUnitOfWork	implements
EF Core query projections	IReadModelQueries	implements
EfRepository	EduHubDbContext	query / track
EfUnitOfWork	EduHubDbContext	SaveChanges / transaction

CP-05 | Course authoring and media integration

Create/edit, upload, process, submit and moderate a course.



READING THE COMPONENT VIEW

- Instructor ownership is enforced before course changes or upload grants; published courses require moderation and media readiness.
- Video processing/compression is required, but its concrete worker/provider is not specified in the brief.

Recommended: direct-to-Blob time-limited upload grants and a replaceable IMediaProcessor implementation.

CP-05 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
EduHub.Web	Instructor CourseBuilder UI	COMPONENT
EduHub.Web	Admin Moderation UI	COMPONENT
EduHub.Services	ICourseService	PORT
EduHub.Services	IMediaService	PORT
EduHub.Services	IModerationService	PORT
EduHub.Services	IMediaProcessor	PORT
EduHub.Services	IMediaStorage	PORT
EduHub.Services	ICourseRepository	PORT

INTERACTION INVARIANTS

- Instructor ownership is enforced before course changes or upload grants; published courses require moderation and media readiness.
- Video processing/compression is required, but its concrete worker/provider is not specified in the brief.

IMPLEMENTATION GUIDANCE

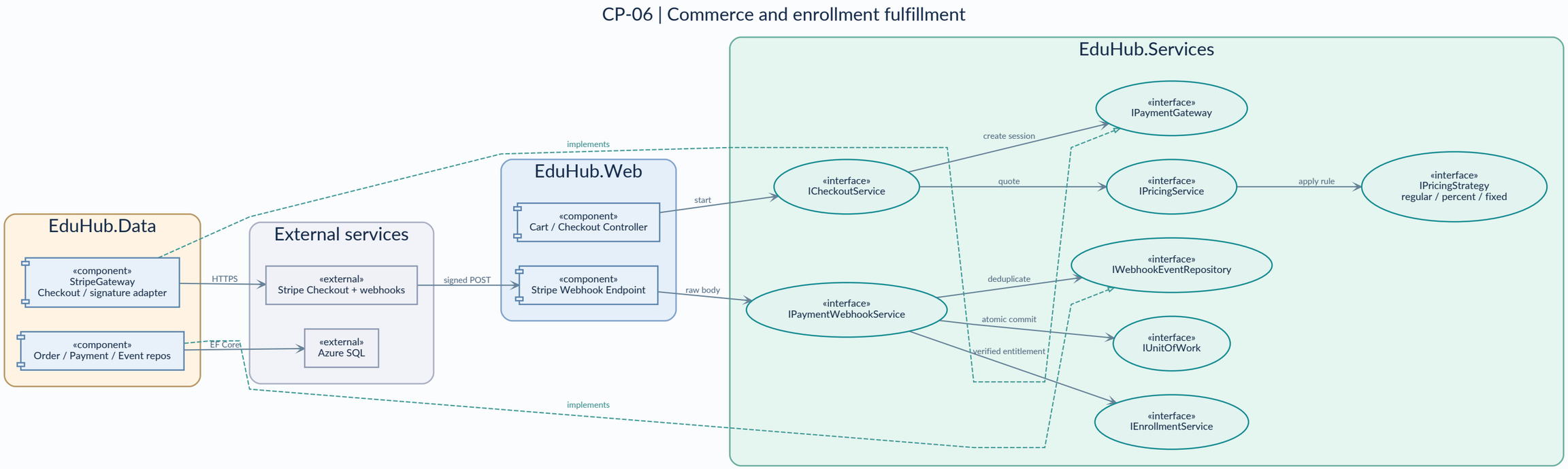
Recommended: direct-to-Blob time-limited upload grants and a replaceable IMediaProcessor implementation.

MODELED EDGES

SOURCE	TARGET	ROLE
Instructor CourseBuilder UI	ICourseService	save draft
Instructor CourseBuilder UI	IMediaService	upload request
Admin Moderation UI	IModerationService	review
IModerationService	ICourseService	approve/reject
ICourseService	ICourseRepository	persists course

CP-06 | Commerce and enrollment fulfillment

Separate checkout initiation from authoritative Stripe webhook fulfillment.



READING THE COMPONENT VIEW

- Only a verified, idempotent server-side payment event may transition a paid order and grant course enrollment.
- Order totals are recomputed server side. Store event IDs and enforce uniqueness; return a safe response for retries.

Architecture callout: server-side signed events are authoritative; the browser success redirect is informational only.

CP-06 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
EduHub.Web	Cart / Checkout Controller	COMPONENT
EduHub.Web	Stripe Webhook Endpoint	COMPONENT
EduHub.Services	ICheckoutService	PORT
EduHub.Services	IPricingService	PORT
EduHub.Services	IPricingStrategy / regular / percent / fixed	PORT
EduHub.Services	IPaymentGateway	PORT
EduHub.Services	IPaymentWebhookService	PORT
EduHub.Services	IEnrollmentService	PORT

INTERACTION INVARIANTS

- Only a verified, idempotent server-side payment event may transition a paid order and grant course enrollment.
- Order totals are recomputed server side. Store event IDs and enforce uniqueness; return a safe response for retries.

IMPLEMENTATION GUIDANCE

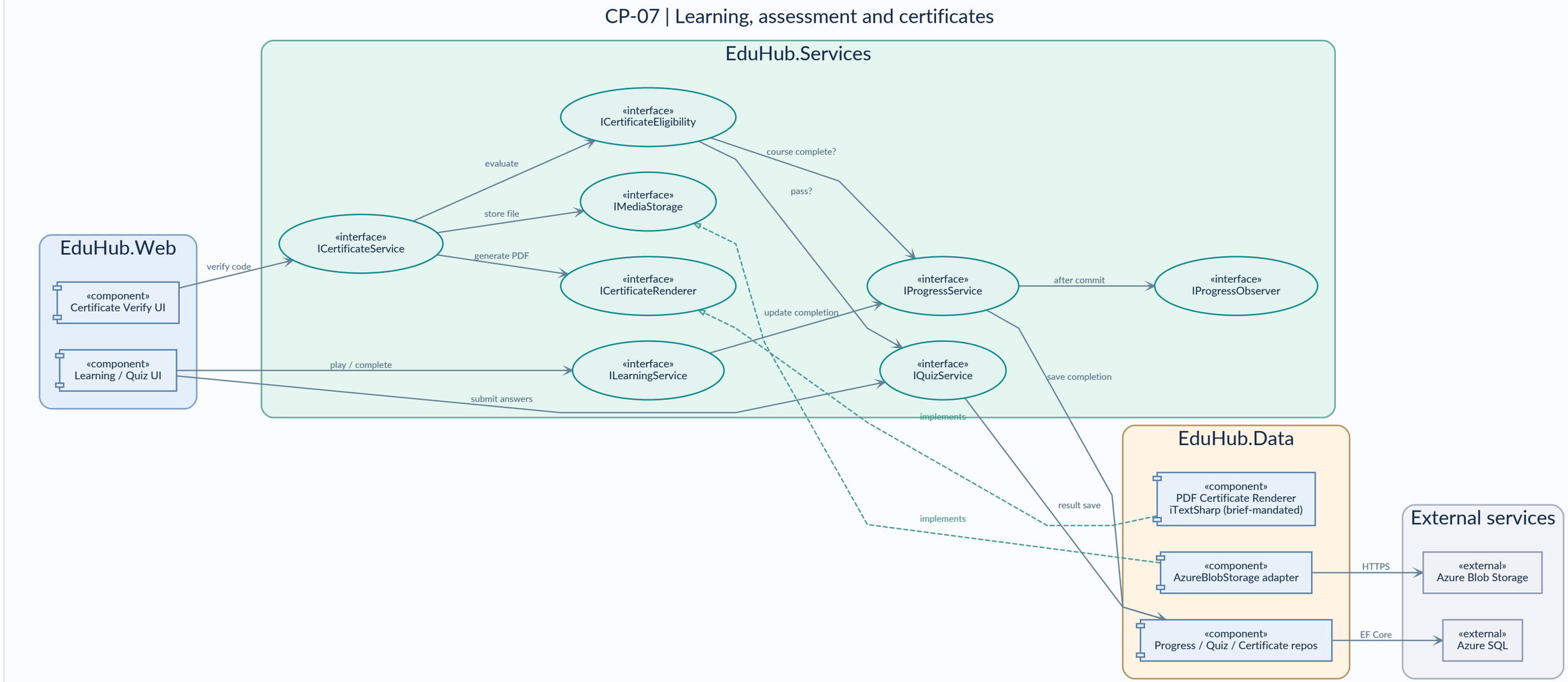
Architecture callout: server-side signed events are authoritative; the browser success redirect is informational only.

MODELED EDGES

SOURCE	TARGET	ROLE
Cart / Checkout Controller	ICheckoutService	start
ICheckoutService	IPricingService	quote
IPricingService	IPricingStrategy	apply rule
ICheckoutService	IPaymentGateway	create session
StripeGateway	IPaymentGateway	implements

CP-07 | Learning, assessment and certificates

Progress writes, quiz results, certificate issuance and storage.



READING THE COMPONENT VIEW

- Student progress and quiz attempts are persisted before emitting Observer notifications or checking certificate eligibility.
- Certificate verification uses a persistent unique code; duplicate issuance and repeated events must be safe.

The brief names iTextSharp; licensing and modern replacement remain an explicit team approval decision.

CP-07 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
EduHub.Web	Learning / Quiz UI	COMPONENT
EduHub.Web	Certificate Verify UI	COMPONENT
EduHub.Services	ILearningService	PORT
EduHub.Services	IProgressService	PORT
EduHub.Services	IQuizService	PORT
EduHub.Services	ICertificateEligibility	PORT
EduHub.Services	ICertificateService	PORT
EduHub.Services	IProgressObserver	PORT

INTERACTION INVARIANTS

- Student progress and quiz attempts are persisted before emitting Observer notifications or checking certificate eligibility.
- Certificate verification uses a persistent unique code; duplicate issuance and repeated events must be safe.

IMPLEMENTATION GUIDANCE

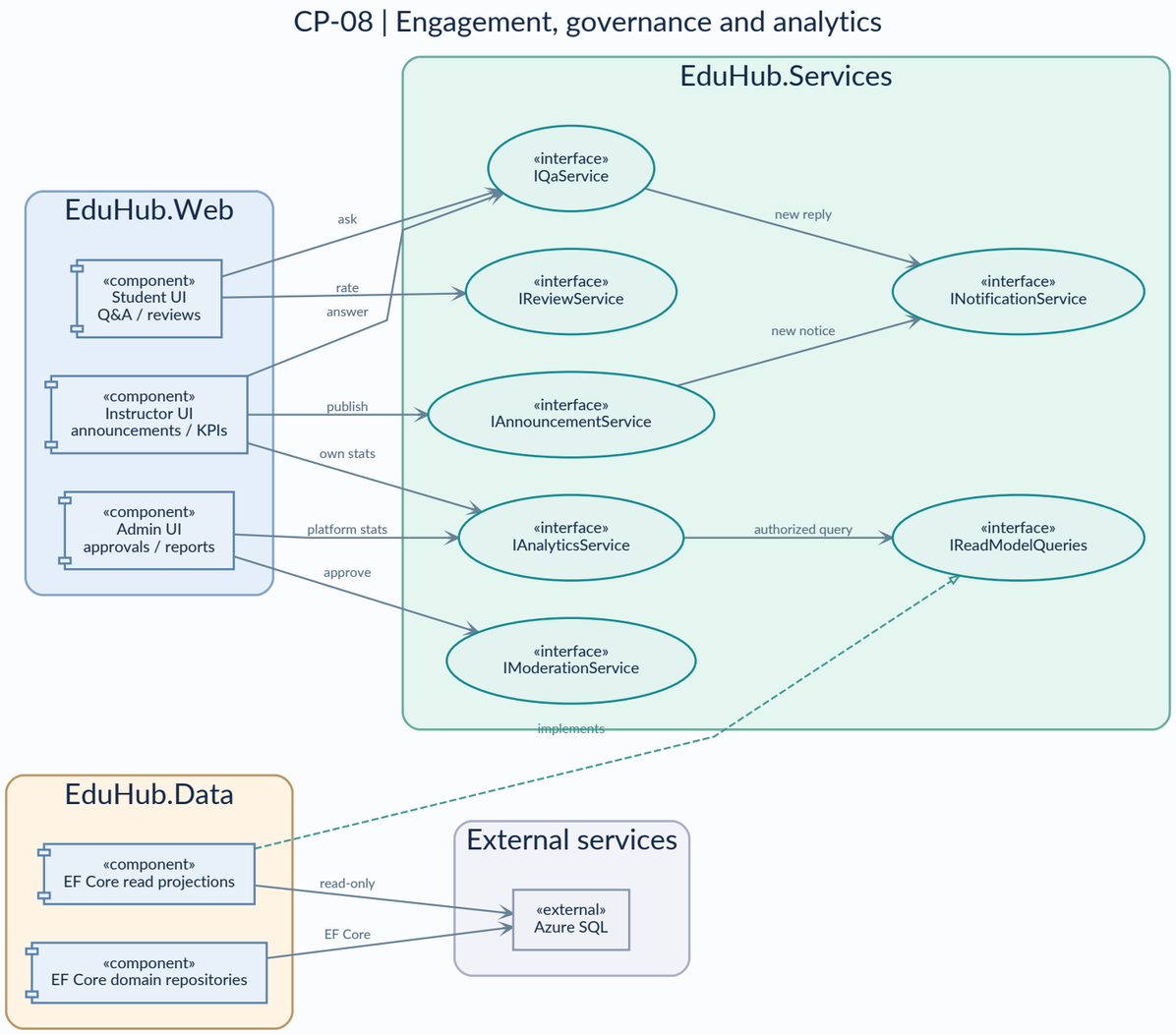
The brief names iTextSharp; licensing and modern replacement remain an explicit team approval decision.

MODELED EDGES

SOURCE	TARGET	ROLE
Learning / Quiz UI	ILearningService	play / complete
ILearningService	IProgressService	update completion
Learning / Quiz UI	IQuizService	submit answers
IQuizService	Progress / Quiz / Certificate repos	result save
IProgressService	Progress / Quiz / Certificate repos	save completion

CP-08 | Engagement, governance and analytics

Questions, reviews, announcements, moderation and read-only reporting.



READING THE COMPONENT VIEW

- Reports are built from read models but still enforce instructor ownership and admin-only access where relevant.
- Notifications may initially be in-process; an external message broker or email provider is not required by the project brief.

Admin and instructor dashboards use Chart.js; reporting definitions and scopes should follow the access matrix.

CP-08 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
EduHub.Web	Student UI / Q&A; / reviews	COMPONENT
EduHub.Web	Instructor UI / announcements / KPIs	COMPONENT
EduHub.Web	Admin UI / approvals / reports	COMPONENT
EduHub.Services	IQaService	PORT
EduHub.Services	IReviewService	PORT
EduHub.Services	IAnnouncementService	PORT
EduHub.Services	IModerationService	PORT
EduHub.Services	IAnalyticsService	PORT

INTERACTION INVARIANTS

- Reports are built from read models but still enforce instructor ownership and admin-only access where relevant.
- Notifications may initially be in-process; an external message broker or email provider is not required by the project brief.

IMPLEMENTATION GUIDANCE

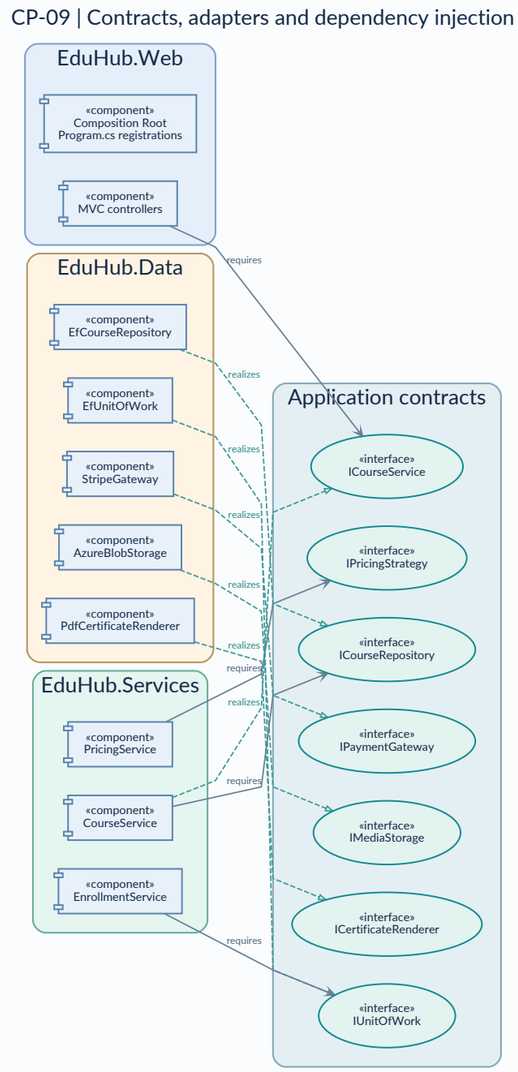
Admin and instructor dashboards use Chart.js; reporting definitions and scopes should follow the access matrix.

MODELED EDGES

SOURCE	TARGET	ROLE
Student UI	IQaService	ask
Student UI	IReviewService	rate
Instructor UI	IQaService	answer
Instructor UI	IAnnouncementService	publish
Instructor UI	IAnalyticsService	own stats

CP-09 | Contracts, adapters and dependency injection

UML interfaces and implementation classes across project boundaries.



READING THE COMPONENT VIEW

- Interfaces keep MVC/controllers and business policies independent of provider SDKs and EF Core types.
- The Web composition root may reference Data to register concrete implementations; normal controllers depend on service abstractions.

Provider adapter placement under EduHub.Data/Integrations is a three-project compromise, pending approval of a separate Infrastructure project.

CP-09 | Design specification

Responsibilities, contracts and architectural guardrails for the preceding vector figure.

SELECTED COMPONENT INVENTORY

BOUNDARY	COMPONENT / PUBLIC CONTRACT	TYPE
EduHub.Web	Composition Root / Program.cs registrations	COMPONENT
EduHub.Web	MVC controllers	COMPONENT
Application contracts	ICourseService	PORT
Application contracts	ICourseRepository	PORT
Application contracts	IUnitOfWork	PORT
Application contracts	IPaymentGateway	PORT
Application contracts	IMediaStorage	PORT
Application contracts	ICertificateRenderer	PORT

INTERACTION INVARIANTS

- Interfaces keep MVC/controllers and business policies independent of provider SDKs and EF Core types.
- The Web composition root may reference Data to register concrete implementations; normal controllers depend on service abstractions.

IMPLEMENTATION GUIDANCE

Provider adapter placement under EduHub.Data/Integrations is a three-project compromise, pending approval of a separate Infrastructure project.

MODELED EDGES

SOURCE	TARGET	ROLE
MVC controllers	ICourseService	requires
CourseService	ICourseRepository	requires
EnrollmentService	IUnitOfWork	requires
PricingService	IPricingStrategy	requires
EfCourseRepository	ICourseRepository	implements

Public service interfaces

Logical service surfaces offered by EduHub.Services to its MVC consumers.

APPLICATION-LEVEL CONTRACTS

INTERFACE	PROVIDED CAPABILITIES	MAIN CONSUMERS
ICourseService	Create draft, edit content, submit for approval, browse published catalog.	Public / Instructor controllers
IModerationService	Review submission, approve/reject, manage categories and platform policies.	Admin controllers
IEnrollmentService	Enroll in free course; grant paid entitlement after verified fulfillment.	Learning + Commerce services
ILearningService	Authorize learning session, manage notes/bookmarks, retrieve lessons.	Student controllers
IProgressService	Mark completion, compute progress, raise post-commit progress event.	Learning and Certificates
IQuizService	Define quizzes, grade attempts, enforce pass thresholds.	Instructor / Student controllers
ICheckoutService	Quote cart, validate discounts, create pending order and checkout session.	Checkout controller
IPaymentWebhookService	Verify/reconcile event, update payment, fulfill order idempotently.	Webhook controller
ICertificateService	Check eligibility, issue unique certificate, verify public code.	Student + public verification
IAnalyticsService	Authorized instructor KPIs and platform reporting projections.	Instructor / Admin controllers
INotificationService	Create in-app alerts from committed domain events.	Services / UI

Exact method names, DTOs, input validation and exception types are not defined in the original brief; treat these as proposed service boundaries pending implementation review.

Infrastructure ports and adapters

Interface-first integration boundaries for the three-project implementation.

PERSISTENCE AND EXTERNAL-PROVIDER CONTRACTS

PORT	IMPLEMENTATION CANDIDATE	GUARDRAIL
ICourseRepository / IRepository	EF Core CourseRepository / EfRepository	Domain-specific queries; avoid open-ended CRUD over all aggregates.
IUnitOfWork	EfUnitOfWork + EduHubDbContext	Atomic related writes; commit once for a business operation.
IReadModelQueries	EF Core no-tracking projections	Filter by user/role before returning dashboard data.
IPaymentGateway	StripeGateway	Provider SDK stays in adapter; server-side checkout session.
IWebhookEventRepository	EF Core WebhookEventRepository	Unique provider event ID; safe duplicate detection.
IMediaStorage	AzureBlobStorage	Private media; scoped upload/read SAS after ownership/entitlement.
IMediaProcessor	Video compression/thumbnail worker adapter	Required capability; concrete processor not specified in brief.
ICertificateRenderer	PdfCertificateRenderer (iTextSharp per brief)	PDF content controlled; licensing and supported alternative review.
IPricingStrategy	Regular, Percentage, Fixed pricing rules	No user-controlled totals; choose strategy by validated coupon/policy.
IProgressObserver	Notification / Eligibility event handlers	Notify after commit; make handler side effects idempotent.

ADAPTER PLACEMENT

A practical three-project solution may place EF Core and provider integrations in EduHub.Data/Integrations. This is a proposal, not a source requirement. Moving non-database adapters to an additional Infrastructure project is optional and requires team approval.

Dependency and ownership rules

Code dependency constraints and the reason each architectural boundary exists.

ALLOWED COMPILE-TIME REFERENCES

FROM PROJECT	MAY REFERENCE	MUST NOT CONTAIN
EduHub.Web	EduHub.Services; EduHub.Data only in Program.cs/DI registration (proposal).	SQL queries inside controllers; business pricing; raw Stripe fulfillment logic.
EduHub.Services	Its own domain types/interfaces; shared BCL and pure libraries.	MVC-specific HttpContext/ActionResult; Stripe/Azure SDKs; concrete DbContext.
EduHub.Data	EduHub.Services contracts; EF Core, Identity persistence, provider SDKs.	Controller/view model rendering; domain decision-making without Services.
External SaaS	No source project references; HTTPS integration only.	No circular references with internal assemblies.

COMPONENT OWNERSHIP RULES

- Catalog owns course content and draft validity; Governance owns final approval decisions.
- Commerce owns pricing, order settlement and payment verification; Enrollment owns entitlements.
- Learning owns progress; Assessment owns quiz pass rules; Certificates owns issuance and verification.
- Media owns upload/playback authorization; Blob holds bytes, not authorization decisions.
- Analytics reads approved projections and must not bypass authorization policies.

Pattern implementation and interaction controls

Map all design patterns named by the EduHub project to concrete component responsibilities.

REQUIRED DESIGN PATTERNS

PATTERN	COMPONENT-LEVEL REALIZATION	ACCEPTANCE INVARIANT
N-Tier	Web controllers → Services interfaces; Data implements repositories/adapters.	No business logic buried in MVC views or persistence classes.
Repository	ICourseRepository and other domain ports; EF Core concrete implementations.	Consumer code depends on port, not DbContext.
Unit of Work	IUnitOfWork wrapping EF Core DbContext/transaction.	A paid fulfillment operation cannot partly commit order/enrollment changes.
Strategy	IPricingStrategy implementations for normal / percentage / fixed discounts.	One validated pricing result irrespective of client inputs.
Observer	Progress completion event handled by notification/eligibility subscribers.	Events follow successful commit; safe duplicate delivery.
Dependency Injection	Web composition root binds interfaces to implementations.	Can replace adapters with test doubles in automated tests.

CRITICAL CROSS-COMPONENT CHECKPOINTS

- Paid enrollment: Stripe signature verified → event deduplicated → order/payment/enrollment committed once.
- Video access: check course state and enrollment → issue short-lived scoped access → client uses Video.js.
- Certificate: persisted progress + required quiz passes → unique code → PDF renderer → private Blob asset.
- Moderation: instructor edits draft; only authorized admin can approve; unpublished content is not publicly visible.

Traceability and design review

Connect component views to previously documented workflows, constraints and implementation tasks.

CROSS-DOCUMENT TRACEABILITY

SOURCE CAPABILITY	COMPONENT VIEW(S)	RELATED DESIGN ARTIFACT
Multi-role registration and authorization	CP-01, CP-02, CP-09	Use cases / identity class model
Course builder, upload and approval	CP-03, CP-05	Course state/activity models
Secure videos and enrolled learning	CP-05, CP-07	Video/payment sequences and deployment
Cart, pricing and paid enrollment	CP-06, CP-09	Stripe webhook sequence; payment state model
Quizzes, progress and certification	CP-07	Learning state/sequence/class diagrams
Reviews, Q&A; and announcements	CP-08	Engagement use cases
Admin, instructor analytics and reports	CP-08	Analytics requirements and DFD
Repository, UoW and SQL schema	CP-04, CP-09	Logical & physical schema; class diagrams

APPROVAL DECISIONS

- ADR-CP01: Approve the names/granularity of service interfaces and route conventions.
- ADR-CP02: Decide whether third-party adapters live in EduHub.Data/Integrations or a fourth project.
- ADR-CP03: Select video transcoding and thumbnail processing runtime/provider.
- ADR-CP04: Confirm iTextSharp usage or authorize a supported equivalent after license review.
- ADR-CP05: Confirm post-commit notification mechanism: in-process only versus durable outbox.

References and verification checklist

Primary project evidence and normative/official technical engineering references.

SOURCES

S1 EduHub - Online Learning Platform, 5-page supplied project brief

S2 EduHub - Software Architecture (previous deliverable)

S3 EduHub - UML Class Diagrams (previous deliverable)

S4 EduHub - Logical & Physical Schema and Deployment Diagram

R1 OMG - UML 2.5.1 specification
https://www.omg.org/spec/UML/2.5.1/

R2 Microsoft - N-tier architecture style
https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier

R3 Microsoft - ASP.NET Core dependency injection
https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection

R4 Microsoft - EF Core relationships
https://learn.microsoft.com/en-us/ef/core/modeling/relationships

R5 Stripe - Checkout fulfillment
https://docs.stripe.com/checkout/fulfillment

R6 Azure Storage - User delegation SAS for Blob
https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-user-delegation-sas-create-dotnet

DESIGN-QUALITY REVIEW

The nine figures use UML component vocabulary and package boundaries. Every external dependency is outside the application boundary; business services use contracts; paid-course fulfillment, video entitlements, and PDF certificates have explicit responsible components. This document is a design proposal, not generated code or deployed software.